



ORACLE
NetSuite

Email Capture Plug-in Guide

Version 2015 Release 1

November 19, 2025



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

Email Capture Plug-in Overview	1
Email Capture Process Flow	1
Developing an Email Capture Plug-in Implementation	2
Create a Plug-in Implementation Script File	3
Add the Plug-in Implementation	4
Test the Plug-in Implementation	6
Bundle the Plug-in Implementation	6
Administering an Email Capture Plug-in Implementation	7
Enable Features for an Email Capture Plug-in Implementation	7
Install an Email Capture Plug-in Bundle	7
Enable the Email Capture Plug-in Implementation	8
Create an Email Alias and Set Up Forwarding	9
Email Capture Plug-in Interface Description	10
process(email)	10
Email	11
Address	16
Attachment	18

Email Capture Plug-in Overview

Use the Email Capture Plug-in to define NetSuite behavior based on the properties and contents of an email message. When NetSuite receives an email message sent to an address associated with an implementation of the plug-in, it runs the business logic defined in the implementation.

 **Note:** Only SuiteScript 1.0 is supported for use with the Email Capture Plug-in.

For example, you can use SuiteScript and an implementation of the Email Capture Plug-in to process invoices, automate task creation and assignment, or automate escalations through email.

 **Important:** Don't use SuiteScript to create cases. If SuiteScript sends an email message to a case capture email address, NetSuite doesn't create a case. This limitation prevents your email system from malfunctioning due to an infinite loop of email messages.

In an Email Capture Plug-in implementation, you can define how the plug-in implementation works based on email properties, including header information, body text, and attachments. Depending on your requirements, you can create several plug-in implementations for different NetSuite functionality. NetSuite generates an email address for each plug-in implementation so you can send email messages for processing.

Solution providers create implementations of the plug-in to define business logic based on the properties and contents of an email message. You can bundle and distribute plug-in implementations as SuiteApps. NetSuite account administrators can install the SuiteApp, and then users can send email messages to the plug-in implementation, which processes the email message and runs the business logic defined in the plug-in implementation.

 **Important:** The plug-in doesn't capture email messages that NetSuite generates and sends to its own plug-in email address, such as saved search and notification email messages. This limitation prevents your NetSuite email system from malfunctioning due to an infinite loop of email messages. To see those email messages, use a different recipient that is not connected to a capture plug-in.

For information about the Email Capture Plug-in, see the following topics:

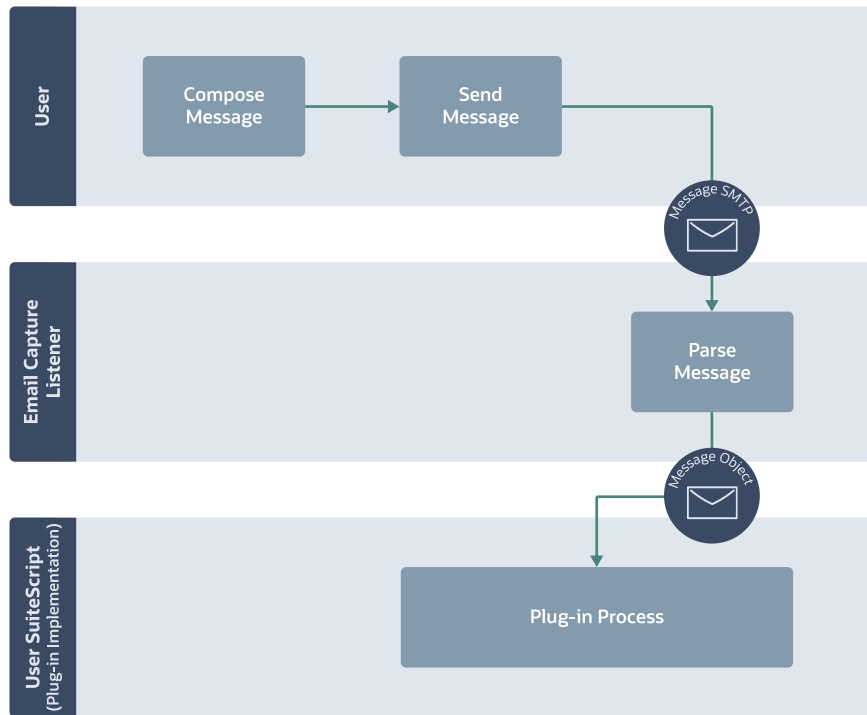
NetSuite Role	For more information, see ...
All roles	Email Capture Process Flow
Developer	Developing an Email Capture Plug-in Implementation
Administrator	Administering an Email Capture Plug-in Implementation

Email Capture Process Flow

The following steps describe the processing of an email message with an Email Capture Plug-in implementation:

- User sends an email message to the email address associated with the plug-in implementation, found on the Manage Plug-ins page.

- NetSuite receives the message and sends it to the appropriate plug-in implementation as an object. You can have multiple Email Capture Plug-in implementations active in a one account.
- The plug-in implementation processes the email message using the business logic you defined in the `process(email)` interface function of the Email Capture Plug-in implementation.



Developing an Email Capture Plug-in Implementation

One Email Capture Plug-in implementation can process different types of email messages, or you can use a separate Email Capture Plug-in implementation for each type of email message. To create a plug-in implementation, use a developer account to develop and test it.

You can use SuiteBundler to bundle the plug-in objects and distribute them to other NetSuite accounts. NetSuite administrators use the bundle to install and enable the plug-in implementation. For more information about administration tasks for an Email Capture Plug-in bundle, see [Administering an Email Capture Plug-in Implementation](#).

The following table describes the basic steps in developing a single plug-in implementation of the Email Capture Plug-in:

Step	Description
Enable features	Enable the Server SuiteScript feature. See Enable Features for an Email Capture Plug-in Implementation .
Create script file	Create the script file that contains the Email Capture Plug-in implementation.

Step	Description
	See Create a Plug-in Implementation Script File .
Add the plug-in implementation	Add the plug-in implementation using the script file and any utility files that you created to the development account. See Add the Plug-in Implementation .
Test the plug-in implementation	Verify that the plug-in implementation works as expected. See Test the Plug-in Implementation .
Bundle the plug-in implementation (Optional)	Bundle the plug-in implementation and any other required objects to distribute to other NetSuite accounts. See Bundle the Plug-in Implementation .

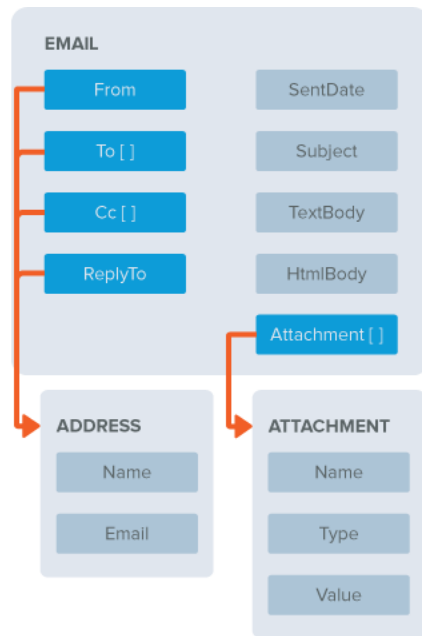
Create a Plug-in Implementation Script File

You can use the SuiteCloud IDE, another JavaScript IDE, or a text editor to create a JavaScript file with the business logic for your Email Capture Plug-in implementation script file.

The following table describes the function and objects available in an Email Capture Plug-in implementation script file:

Function Object	Description
process(email) interface function	Interface function that contains the implementation of the business logic of the Email Capture Plug-in. You can use JavaScript and SuiteScript API functions to define the business logic. This function is required for all Email Capture Plug-in implementations and NetSuite calls it automatically when someone sends a message to the email address associated with the plug-in implementation. For more information about how NetSuite uses this function, see Email Capture Process Flow .
Email object	Object that represents an email message sent to the Email Capture Plug-in implementation. Use the methods to retrieve the headers, date sent, subject, body, and attachment properties of an email message.
Address object	Object that represents data from the from, to, cc, or reply-to headers in an email message sent to an Email Capture Plug-in implementation.
Attachment object	Object that represents an attachment in an email message sent to an Email Capture Plug-in implementation. Each Attachment object contains properties for the file name, attachment type, and the value of the attachment file.

The following diagram shows the object model for the [Email](#) interface input object:



Rules and Guidelines

Use the following rules and guidelines when creating the plug-in implementation script file:

- The plug-in script file can have any name, provided it contains an implementation of the interface function.
- If you want to create utility files with helper functions for the main implementation file, you can include them when you create the plug-in implementation for the Email Capture Plug-in in NetSuite. See [Add the Plug-in Implementation](#).
- You can use SuiteScript API functions in a plug-in implementation, for example, [record.load\(options\)](#) and [search.create\(options\)](#). However, using these APIs may slow the performance of the plug-in implementation, so limit the use of these APIs when possible. In general, searching for records yields better performance than loading them.
Governance limits apply to these functions too. The plug-in script file lets you use up to 1000 usage units with SuiteScript. For more information, see the help topic [SuiteScript 2.x API Governance](#).
- For security reasons, you can't send an email message to the Email Capture Plug-in from within NetSuite. You must use an external mail client. For example, you can't set up a script to send an email to the Email Capture Plug-in when a customer is created.

Add the Plug-in Implementation

When you finish creating the plug-in implementation script file, create a new plug-in implementation in your development account. When you create the plug-in implementation, you upload the script file and other utility files as needed. You can bundle this implementation to distribute it to other NetSuite accounts.

To create a new plug-in implementation:

1. Review the interface description for your core plug-in.
2. Create a JavaScript file for the implementation. Use this file to define the logic for the methods listed in the interface description.

3. In the UI, click Customization > Plug-ins > Plug-in Implementations > New.
4. In the **Script File** field, open the script file or add a new one.
5. Click **Create Plug-in Implementation**.
6. On the Select Plug-in Type page, click the **Email Capture** link.
7. On the Plug-in Implementation page, enter the following information.

Option	Description
Name	User-friendly name for the implementation. The plug-in implementation appears in the following locations: ■ Manage Plug-ins page. Page used by administrators to enable or disable the plug-in implementation in their account. ■ Bundle Builder. Select this name in the Bundle Builder to distribute the plug-in implementation to other accounts.
ID	Internal ID of the implementation for scripting. If you leave this field blank, NetSuite provides an ID when you click Save .
Status	Current status for the implementation. Choose Testing to make the implementation available only to the owner, or Released to make it available to all accounts in production.  Note: The status must be set to Released for the plug-in to work with email messages coming from outside of NetSuite. When the status is set to Testing, only the script owner can use the plug-in implementation. Since email messages arrive from outside of NetSuite, it can't tell who sent them, so the plug-in implementation <i>won't work</i> if the status is set to Testing.
Log Level	Logging level for the script. Select Debug , Audit , Error , or Emergency . These messages appear on the Execution Log subtab for the plug-in implementation.
Execute As Role	Role that the script runs as. The Execute As Role field lets you control permissions and restrictions of the executing script. Select Current Role to run the script using the permissions of the user who is currently logged in.  Note: You can create the custom role for testing, provided it has the SuiteScript permission. You can bundle this custom role with the plug-in implementation. See the help topics Test the Plug-in Implementation and Bundle the Plug-in Implementation.
Description	Optional description of the implementation, displayed on the Plug-In Implementations page.
Owner	User account that owns the implementation. By default, it's the name of the logged-in user.
Inactive	Indicates that the plug-in implementation isn't running in the account. Inactivate a plug-in implementation to temporarily disable it for testing.



Important: You can't run an implementation as Administrator. You must select a custom role in the **Execute As Role** field. Note that System Administrator is a custom role.

8. On the **Scripts** subtab, in the **Implementation** list, change the JavaScript file that contains the implementation of the plug-in, if required.
9. On the **Scripts** subtab, in the **Library Script File** list, select any utility or library files that plug-in script file needs.

10. On the **Unhandled Errors** subtab, specify who to notify if script errors occur.
 - To notify the user who is logged in and running the script, check the **Notify Current User** box.
 - To notify the script owner, check the **Notify Script Owner** box. The box is checked by default.
 - To notify all administrators, check the **Notify All Admins** box.
 - To notify a group, select the group to notify. Only groups set up in Lists > Relationships > Groups are available.
 - Enter individual email addresses in the **Notify Emails** field. Separate multiple email addresses with a semicolon.
11. Click **Save**.

You can see the list of implementations by going to Customization > Plug-ins > Plug-in Implementations.



Note: You can use SuiteCloud Development Framework (SDF) to manage Email Capture Plug-in implementations as part of file-based customization projects. For information about SDF, see the help topic [SuiteCloud Development Framework](#).

You can use Copy to Account to copy one Email Capture Plug-in implementation to another account. When you edit an Email Capture Plug-in implementation, you'll see a **Copy to Account** option in the upper right corner. For information about Copy to Account, see the help topic [Copy to Account](#).

Debugging a Core Plug-in Implementation

Test the Plug-in Implementation

To test an Email Capture Plug-in implementation, perform the following tasks:

- Enable the implementation. Go to the Manage Plug-ins page in your development account to enable the plug-in implementation for testing. The Manage Plug-ins page includes the email address associated with the plug-in implementation. See [Enable the Email Capture Plug-in Implementation](#).
- Create a custom role for the plug-in implementation. The **Execute As Role** field specifies the role the plug-in implementation script runs as, and it needs the SuiteScript permission.
For more information about the **Execute As Role** field, see [Add the Plug-in Implementation](#). For more information about custom roles in NetSuite, see the help topic [Customizing or Creating NetSuite Roles](#).
- Send messages to the email address for the plug-in implementation to test the functionality.



Important: You should test your plug-in implementation on every NetSuite version your customers use.

Bundle the Plug-in Implementation

After developing the Email Capture Plug-in implementation, you can distribute the implementation to a production account. SuiteBundler lets you package objects together as bundles, or SuiteApps, for distribution to other accounts. To distribute the Email Capture Plug-in implementation script file, utility files, custom roles, and the plug-in implementation object, create a bundle with SuiteBundler. After you create the bundle, administrators can install it in production accounts.

The following table lists the objects you must include in the bundle and their location on the **Select Objects** page in the Bundle Builder:

Object	Location On Select Objects Page
Plug-in implementation script file Utility files	File Cabinet > Files
Custom roles	Roles > Custom Roles
Plug-in implementation	Plug-ins > Email Capture Plug-in

Administering an Email Capture Plug-in Implementation

After a developer creates and bundles an implementation of an Email Capture Plug-in, you can install it and set it up.

To install and set up an Email Capture Plug-in implementation, complete the following steps:

- [Enable Features for an Email Capture Plug-in Implementation](#)
- [Install an Email Capture Plug-in Bundle](#)
- [Enable the Email Capture Plug-in Implementation](#)
- [Create an Email Alias and Set Up Forwarding](#)

Enable Features for an Email Capture Plug-in Implementation

Before you install an Email Capture Plug-in SuiteApp, make sure that the Server SuiteScript feature is enabled in the account.

To enable features for the Email Capture Plug-in:

1. Choose Setup > Company > Enable Features.
2. On the **SuiteCloud** subtab, enable the Server SuiteScript feature.
3. If needed, check the box and agree to the Terms of Service.
4. Click **Save**.

Install an Email Capture Plug-in Bundle

A developer can create and bundle an implementation of the Email Capture Plug-in for distribution to other NetSuite accounts. An administrator can then install the bundle in the target account.

 **Important:** Before you try to install the bundle, check with your NetSuite Administrator to make sure that the bundle has been shared on your account. See the help topic [Sharing a Bundle](#).

To install an Email Capture Plug-in bundle:

1. Go to Customization > SuiteBundler > Search & Install Bundles.
2. On the Install Bundle page, select **Advanced** and choose **Production Account** in the **Location** dropdown.
3. In the **Keywords**, enter Email Capture Bundle and click **Enter**.
4. Select the bundle from the list.
5. Click **Install** for the bundle.



Important: To avoid duplicate objects, select **Replace Existing Object** if you're prompted during install.

After you start installing a bundle, you can continue working in NetSuite as the bundle installs.

To check on the progress of the installation, go to Customization > SuiteBundler > Search & Install Bundles > List. If installation is not complete, the Status column shows the progress as a percentage. Click **Refresh** to update the status. When the installation is complete, you'll see a green check mark.



Note: If you don't see an **Install** button, the bundle may not be shared with your account. To get access to the bundle, contact the developer or NetSuite Technical Support.

Enable the Email Capture Plug-in Implementation

A user with the Administrator role enables the Email Capture Plug-in implementation to activate it in your account.

To enable the Email Capture Plug-in implementation:

1. Go to: Customization > Plug-ins > Manage Plug-ins.
2. Under Email Capture Plugin, check the **Log Email Details** box next to the name of the plug-in implementation.

WARNING
Plug-ins are generally designed to modify or supplement NetSuite business logic. When you enable a plug-in, standard NetSuite business processes may no longer run as expected, especially if you enable a plug-in developed by a third party. Therefore, make sure that you understand the impact on your NetSuite account before enabling a plug-in and ensure that use of the plug-in meets all of your security, regulatory, and other compliance obligations, including Payment Card Industry (PCI) Data Security Standards.

Manage Plug-In Implementations

Save **Cancel**

Email Capture Plugin
IMPLEMENTATIONS ☒ LOG EMAIL DETAILS emails:123456.383.6e873a723f@123456.email.netsuite.com

3. Click **Save**.



Note: NetSuite automatically generates a unique capture email address for each plug-in implementation. You'll see the email address next to the plug-in implementation name on the Manage Plug-in Implementations page. The plug-in implementation executes when users send an email message to the address associated with the plug-in implementation.

Create an Email Alias and Set Up Forwarding

NetSuite generates an email address for the plug-in implementation. Users can send email messages to the implementation's email address for processing. However, you may want to create an email alias on your company mail server to hide the email address required by the plug-in implementation.

For example, if your account ID is 123456, the NetSuite generated email address for a plug-in implementation would look something like `emails.123456.xxx.xxx@1234567.email.netsuite.com`. The plug-in implementation uses that email address to send invoice records from NetSuite. You can create an email alias named `invoices@example.com` on your company mail server that forwards to the plug-in implementation email address.

The process to set up an email alias and forwarding varies by provider. Refer to the documentation for your email software for instructions.

You can then let customers know to send their invoices to `invoices@example.com`.

Email Capture Plug-in Interface Description

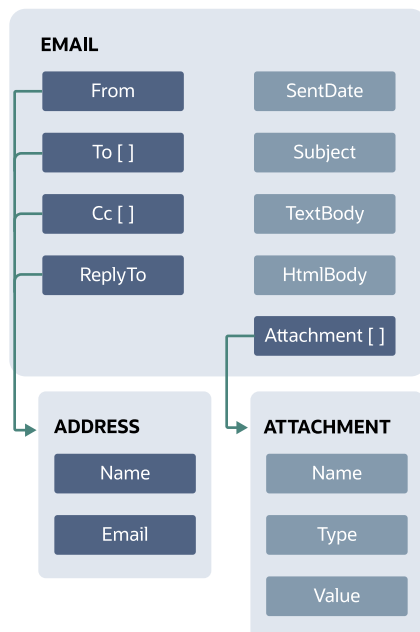
Interface Function

The Email Capture plug-in interface includes the following functions:

Function	Description
process(email)	Interface function that contains the implementation of the business logic of the Email Capture plug-in. You can use JavaScript and SuiteScript API functions to define the business logic. This function is required for all Email Capture plug-in implementations and is called automatically by NetSuite when an email is sent to the email address associated with the plug-in implementation. For more information about how NetSuite uses this function, see Email Capture Process Flow.

Email Capture Plug-in Object Model

The following diagram shows the Email Capture plug-in object model:



[process\(email\)](#)

Function Declaration void process(Email email)

Type Interface function

Description Interface function that contains the implementation of the business logic of the Email Capture plug-in. NetSuite calls this function when it receives an email message sent to the email address associated with the plug-in implementation. See [Email Capture Process Flow](#).

Use the methods available to the [Email](#) object to retrieve the properties of the email message, such as sender and recipient fields as [Address](#) objects, subject and body fields, sent date, and attachments as [Attachment](#) objects.

Returns void

Parameters ■ [Email](#)

Example

```

1  function process(email) {
2      ...
3      // log all information in email
4      var fromAddress = email.getFrom();
5      logAddress('from', fromAddress);
6
7      var to = email.getTo();
8      for (var indexTo in to)
9      {
10         logAddress('to', to[indexTo]);
11     }
12     var cc = email.getCc();
13     for (var indexCc in cc)
14     {
15         logAddress('cc', cc[indexCc]);
16     }
17
18     logAddress('replyTo', email.getReplyTo());
19     nlapiLogExecution('DEBUG', 'sent', email.getSentDate());
20     nlapiLogExecution('DEBUG', 'subject', email.getSubject());
21     nlapiLogExecution('DEBUG', 'text body', email.getTextBody());
22     nlapiLogExecution('DEBUG', 'html body', email.getHtmlBody());
23
24     var attachments = email.getAttachments();
25     for (var indexAtt in attachments)
26     {
27         nlapiLogExecution('DEBUG', 'attachments', attachments[indexAtt]);
28     }
29
30     function logAddress(label, address)
31     {
32         if (address)
33         {
34             nlapiLogExecution('DEBUG', 'Email - ' + label + ': ' + address.getName() + ', ' + address.getEmail());
35         }
36     }
37     ...
38     ...
39 }

```

Email

Type Interface input object

Description Object that represents an email message sent to the Email Capture plug-in implementation. Use the methods available to the Email object to retrieve the following properties:

- Sender and recipient headers. The **From**, **To**, **Cc**, and **ReplyTo** properties of the email message as an [Address](#) object. See [getFrom\(\)](#), [getTo\(\)](#), [getCc\(\)](#), and [getReplyTo\(\)](#).
- Date. Date the email message was sent. See [getSentDate\(\)](#).
- Subject field and email body. The value of the subject field and the value of the email body as a text string or HTML string. See [getSubject\(\)](#), [getTextBody\(\)](#), and [getHtmlBody\(\)](#).
- Attachment. An array of all attachments as [Attachment](#) objects. See [getAttachments\(\)](#).

Methods	■ getFrom() ■ getTo() ■ getCc() ■ getReplyTo() ■ getSentDate() ■ getSubject() ■ getTextBody() ■ getHtmlBody() ■ getAttachments()
Parent Object(s)	—
Child Object(s)	■ Address ■ Attachment

getFrom()

Function Declaration	Address getFrom()
Type	Object method
Description	Returns Address object that represents the from header on an email message sent to an Email Capture plug-in implementation.
Returns	Address []
Since	2015.1
Input Parameters	None.
Parent object	■ Email

Example

```

1  function process(email) {
2      ...
3      var fromAddress = email.getFrom();
4      logAddress('from', fromAddress);
5      ...
6  }
```

getTo()

Function Declaration	Address getTo()
Type	Object method
Description	Returns an array of Address objects that represent the to header on an email message sent to an Email Capture plug-in implementation.
Returns	Address
Since	2015.1
Input Parameters	None.

Parent object ■ [Email](#)

Example

```

1  function process(email) {
2      ...
3      var to = email.getTo();
4      for (var indexTo in to)
5      {
6          logAddress('to', to[indexTo]);
7      }
8      ...
9  }

```

getCc()

Function Declaration	Address getCc()
Type	Object method
Description	Returns an array of Address objects that represent the cc header on an email message sent to an Email Capture plug-in implementation.
Returns	Address
Since	2015.1
Input Parameters	None.
Parent object	■ Email

Example

```

1  function process(email) {
2      ...
3      var cc = email.getCc();
4      for (var indexCc in cc)
5      {
6          logAddress('cc', cc[indexCc]);
7      }
8      ...
9  }

```

getReplyTo()

Function Declaration	Address getReplyTo()
Type	Object method
Description	Returns an Address object that represents the reply-to header on an email message sent to an Email Capture plug-in implementation.
Returns	Address
Since	2015.1
Input Parameters	None.

Parent object ■ [Email](#)

Example

```

1 | function process(email) {
2 |     ...
3 |     var replyAddr = email.getReplyTo();
4 |     ...
5 | }
```

getSentDate()

Function Declaration	Date getSentDate()
Type	Object method
Description	Returns a JavaScript Date object that represents the sent date on an email message sent to an Email Capture plug-in implementation.
Returns	Date
Since	2015.1
Input Parameters	None.
Parent object	■ Email

Example

```

1 | function process(email) {
2 |     ...
3 |     var date = email.getSentDate();
4 |     ...
5 | }
```

getSubject()

Function Declaration	string getSubject()
Type	Object method
Description	Returns a string that represents the subject field in an email message sent to an Email Capture plug-in implementation. Returns an empty string if the email message does not contain text in the subject field.
Returns	string
Since	2015.1
Input Parameters	None.
Parent object	■ Email

Example

```

1 | function process(email) {
```

```

2 |     ...
3 |     var subject = email.getSubject();
4 |     ...
5 | }

```

getTextBody()

Function Declaration string getTextBody()

Type Object method

Description Returns a string that represents the plaintext part of the body of an email message sent to an Email Capture plug-in implementation. Returns null if there is no text in the body of the email message.

If there is text in the body field, this method returns a string regardless of the format of the email message. For example, if an email message was sent in HTML format, the string contains the text of the body field without any HTML formatting.

Returns string

Since 2015.1

Input Parameters None.

Parent object ■ Email

Example

```

1 | function process(email) {
2 |     ...
3 |     log('text body', email.getTextBody());
4 |     ...
5 | }

```

getHtmlBody()

Function Declaration string getHtmlBody()

Type Object method

Description Returns a string that represents the HTML content included in the body field of an email message sent to an Email Capture plug-in implementation. Returns null if there is no HTML content in the body of the email message.

For example, this method returns null if the body of an email message contains plain text with no formatting.

Returns string

Since 2015.1

Input Parameters None.

Parent object ■ Email

Example

```

1 | function process(email) {

```

```

2      ...
3      log('html body', email.getHtmlBody());
4      ...
5  }

```

getAttachments()

Function Declaration	Attachment[] getAttachments()
Type	Object method
Description	Returns an array of Attachment objects that represent the attachments included in an email message. Returns an empty array of the email message if does not include an attachment. Use the methods available to the Attachment object to get the name, type, and value of each attachment.
Returns	Attachment[]
Since	2015.1
Input Parameters	None.
Parent object	■ Email

Example

```

1  function process(email) {
2      ...
3      var attachments = email.getAttachments();
4      for (var indexAtt in attachments)
5      {
6          logAttachment('att', attachments[indexAtt]);
7      }
8      ...
9  }

```

Address

Type	Object
Description	Object that represents data from the from, to, cc, or reply-to headers in an email message sent to an Email Capture plug-in implementation.
Methods	■ getName() ■ getEmail()
Parent Object(s)	■ Email
Child Object(s)	—

getName()

Function Declaration	string getName()
-----------------------------	------------------

Type	Object method
Description	Returns a string that represents the name associated with an email address in the from, to, cc, or reply-to headers of an email message. Returns null if the address does not contain a name associated with the email address.
Returns	string
Since	2015.1
Input Parameters	None.
Parent object	■ Address

Example

```

1  function process(email) {
2      ...
3      var to = email.getTo();
4      for (var indexTo in to)
5      {
6          logAddress('to', to[indexTo]);
7      }
8      ...
9      function logAddress(label, address)
10     {
11         if (address)
12         {
13             nlapiLogExecution('DEBUG', 'Email - ' + label + ': ' + address.getName() + ', ' + address.getEmail());
14         }
15     }
16     ...
17 }

```

getEmail()

Function Declaration	string getEmail()
Type	Object method
Description	Returns a string that represents the email address in the from, to, cc, or reply-to headers of an email message. The headers must contain an email address.
Returns	string
Since	2015.1
Input Parameters	None.
Parent object	■ Address

Example

```

1  function process(email) {
2      ...
3      var to = email.getTo();
4      for (var indexTo in to)
5      {
6          logAddress('to', to[indexTo]);

```

```

7      }
8      ...
9      function logAddress(label, address)
10     {
11         if (address)
12         {
13             nlapiLogExecution('DEBUG', 'Email - ' + label + ': ' + address.getName() + ', ' + address.getEmail());
14         }
15     }
16     ...
17 }

```

Attachment

Type	Object
Description	Object that represents an attachment in an email message sent to an Email Capture plug-in implementation. Each Attachment object contains properties for the attachment file name, attachment type, and the value of the attachment file.
Methods	■ getName() ■ getType() ■ getValue()
Parent Object(s)	■ Email
Child Object(s)	—

getName()

Function Declaration	string getName()
Type	Object method
Description	Returns the file name for an attachment in an email message.
Returns	string
Since	2015.1
Input Parameters	None.
Parent object	■ Attachment

Example

```

1      function process(email) {
2          ...
3          var attachments = email.getAttachments();
4          for (var indexAtt in attachments)
5          {
6              logAttachment('att', attachments[indexAtt]);
7          }
8          ...
9      }
10     function logAttachment(label, attachment)
11     {

```

```

12     nlapiLogExecution('DEBUG', 'Att - ' + label + ': ' + attachment.getName() + ', ' + attachment.getType());
13     nlapiLogExecution('DEBUG', 'Att - ' + label + ' - value: ' + attachment.getValue());
14 }
15 ...
16 }

```

getType()

Function Declaration	string getType()
Type	Object method
Description	Returns the file type of an attachment in an email message as a string. For example, this method returns PLAINTEXT, PDF, and MISCBINARY for text, PDF, and Microsoft Word files, respectively.
Returns	string
Since	2015.1
Input Parameters	None.
Parent object	■ Attachment

Example

```

1  function process(email) {
2      ...
3      var attachments = email.getAttachments();
4      for (var indexAtt in attachments)
5      {
6          logAttachment('att', attachments[indexAtt]);
7      }
8      ...
9
10     function logAttachment(label, attachment)
11     {
12         nlapiLogExecution('DEBUG', 'Att - ' + label + ': ' + attachment.getName() + ', ' + attachment.getType());
13         nlapiLogExecution('DEBUG', 'Att - ' + label + ' - value: ' + attachment.getValue());
14     }
15     ...
16 }

```

getValue()

Function Declaration	string getValue()
Type	Object method
Description	Returns a text string for a text file attachment or base-64 encoded string for binary attachment types of an email message. You can use getType() to define the behavior of the plug-in implementation depending on the file type of the attachment.
Returns	string
Since	2015.1
Input Parameters	None.
Parent object	■ Attachment

Example

```
1  function process(email) {  
2      ...  
3      var attachments = email.getAttachments();  
4      for (var indexAtt in attachments)  
5      {  
6          logAttachment('att', attachments[indexAtt]);  
7      }  
8      ...  
9  
10     function logAttachment(label, attachment)  
11     {  
12         nlapiLogExecution('DEBUG', 'Att - ' + label + ': ' + attachment.getName() + ', ' + attachment.getType());  
13         nlapiLogExecution('DEBUG', 'Att - ' + label + ' - value: ' + attachment.getValue());  
14     }  
15     ...  
16 }
```